

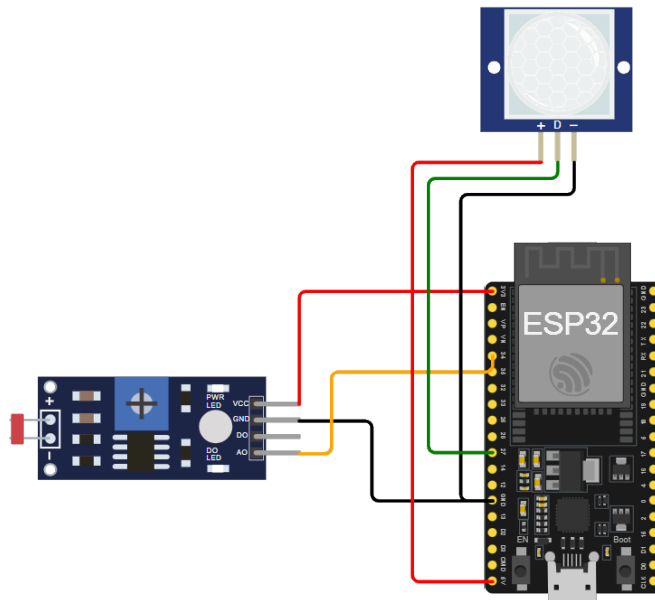
Challenge 1 IoT

Mattia Brianti 10773859

Alex Hathaway 10829946

1 Schema

This setup describes an IoT monitoring node centered using an ESP32 microcontroller, integrated with two sensors: a photoresistor (LDR) and a PIR motion sensor. This configuration simultaneously monitors environmental motion and ambient light levels. The LDR sensor is powered by 3.3V while the PIR sensor is powered by 5V. Both components share a common ground (GND) with the ESP32.



2 Code logic

2.1 Initialization

In this code snippet, the main libraries for wireless communication and low-level system control are imported, and specific hardware pins are assigned to the two sensors. Finally, a broadcast MAC address and a peer-to-peer structure are configured to transmit data to a 'sink' node waiting for instructions. Moreover, constants to convert the signals read by the sensors are defined. Ultimately `TIME_TO_SLEEP` is calculated using the team leader's personal code, as specified, and a conversion factor is defined.

```
1 #include <WiFi.h>
2 #include <esp_now.h>
3 #include "soc/soc.h"
4 #include "soc/rtc_cntl_reg.h"
5
6 // Define PINs
7 #define PIR_PIN 27
```

```

8 #define LDR_PIN 34
9
10 // MAC receiver (Broadcast address for the SINK node)
11 uint8_t broadcastAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
12
13 // These constants should match the photoresistor's "gamma" and "rl10" attributes
14 const float GAMMA = 0.7;
15 const float RL10 = 50;
16
17 esp_now_peer_info_t peerInfo;
18
19 // Parameters for Deep Sleep
20 #define uS_TO_S_FACTOR 1000000 /* Conversion factor for micro seconds to seconds */
21 #define TIME_TO_SLEEP 1.4 /* Computed as (59%50+5)/10 = 1.4

```

2.2 Callback

This code defines the callback function that triggers automatically after the ESP32 attempts to send data via the ESP-NOW protocol. Its primary purpose is to provide feedback on whether the transmission was successful or failed.

```

1 void OnDataSent(const wifi_tx_info_t *mac_addr, esp_now_send_status_t status) {
2     Serial.print("Send Status: ");
3     Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Ok" : "Error");
4 }

```

2.3 Setup

In this section, the system handles the core execution flow while profiling the time spent in each operational phase using the `micros()` functions. First, the microcontroller acquires digital motion data from the PIR sensor and analog light levels from the LDR, converting the raw analog readings into a precise lux value. To minimize power consumption, the Wi-Fi radio remains off during this sensing phase; it is then enabled to transmit the formatted sensor data via the ESP-NOW protocol. Immediately after the message is dispatched, the Wi-Fi module is disabled. Finally, the system logs the exact execution times for each phase via the serial interface before entering a deep sleep state for the predefined interval. These execution times printings are excluded from the second part of the idle phase, since they are inserted in the code only for energy estimation purposes.

```

1 void setup() {
2
3     //--- 1. MEASURE IDLE TIME (Part where the wifi is off) ---
4     unsigned long t_wifi_off_idle_start_1 = micros();
5
6     WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0); // Disable brownout detector
7
8     Serial.begin(115200);
9
10    // Setting up the PIR and LDR pins
11    pinMode(PIR_PIN, INPUT);
12    pinMode(LDR_PIN, INPUT);
13
14    unsigned long t_wifi_off_idle_end_1 = micros();
15    // ---- FINISH 1ST PART OF IDLE TIME MEASUREMENT ----
16
17    // --- 1. MEASURE SENSING TIME ---
18    unsigned long t_sensing_start = micros();
19
20    // Read the PIR and LDR values
21    int motionDetected = digitalRead(PIR_PIN);
22    int analogValue = analogRead(LDR_PIN);
23

```

```

24 // Convert the analog value into lux value
25 float voltage = analogValue / 4095. * 3.3;
26 float resistance = 10000 * voltage / (3.3 - voltage);
27 float luminosity = pow(RL10 * 1e3 * pow(10, GAMMA) / resistance, (1 / GAMMA));
28
29 unsigned long t_sensing_end = micros();
30 //--- SENSING TIME MEASUREMENT END ---
31
32 // --- 2. MEASURE Wi-Fi-ON Time ---
33 unsigned long t_wifi_on_start = micros();
34 Serial.println("Enabling WiFi STA");
35 WiFi.mode(WIFI_STA);
36 esp_now_init();
37
38 // Register the send callback
39 esp_now_register_send_cb(OnDataSent);
40
41 // Peer Registration
42 memcpy(peerInfo.peer_addr, broadcastAddress, 6);
43 peerInfo.channel = 0;
44 peerInfo.encrypt = false;
45
46 // Add peer
47 esp_now_add_peer(&peerInfo);
48
49 // Create the message to send
50 String message;
51 if (motionDetected == HIGH) {
52     message = "MOTION_DETECTED-LUMINOSITY:" + String(luminosity);
53 } else {
54     message = "MOTION_NOT_DETECTED-LUMINOSITY:" + String(luminosity);
55 }
56
57 // --- 2. MEASURE TRANSMISSION TIME ---
58 unsigned long t_tx_start = micros();
59
60 // Send the message via ESP-NOW
61 esp_now_send(broadcastAddress, (uint8_t*)message.c_str(), message.length() + 1);
62
63 unsigned long t_tx_end = micros();
64 //--- TRANSMISSION TIME MEASUREMENT END ---
65
66 Serial.println("Message sent: " + message);
67
68 // Disable Wi-Fi
69 Serial.println("\nDisabling WiFi");
70 WiFi.mode(WIFI_OFF);
71
72 unsigned long t_wifi_on_end = micros();
73 // ---- FINISH Wi-Fi ON Time Measurement ---
74
75 // --- 3. MEASURE IDLE TIME (Wi-Fi OFF to Deep Sleep) ---
76 unsigned long t_wifi_off_idle_start_2 = micros();
77
78 Serial.println("Entering deep sleep now...");
79 esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * uS_TO_S_FACTOR);
80
81 unsigned long t_wifi_off_idle_end_2 = micros();
82
83 // --- TIMING CALCULATIONS & OUTPUT ---
84 unsigned long time_wifi_on = t_wifi_on_end - t_wifi_on_start;
85 unsigned long time_sensing = t_sensing_end - t_sensing_start;

```

```

86 unsigned long time_tx = t_tx_end - t_tx_start;
87 unsigned long time_wifi_off_idle_1 = t_wifi_off_idle_end_1 - t_wifi_off_idle_start_1;
88 unsigned long time_wifi_off_idle_2 = t_wifi_off_idle_end_2 - t_wifi_off_idle_start_2;
89
90 Serial.println("\n--- TIMING BREAKDOWN (in microseconds) ---");
91 Serial.print("Sensing duration: "); Serial.println(time_sensing);
92 Serial.print("Wi-Fi On duration: "); Serial.println(time_wifi_on);
93 Serial.print("Transmission duration: "); Serial.println(time_tx);
94 Serial.print("Idle duration (Wi-Fi OFF - Part 1): "); Serial.println(
    time_wifi_off_idle_1);
95 Serial.print("Idle duration (Wi-Fi OFF - Part 2): "); Serial.println(
    time_wifi_off_idle_2);
96 Serial.print("Total Idle duration (Wi-Fi OFF): "); Serial.println(time_wifi_off_idle_1
    + time_wifi_off_idle_2);
97 Serial.println("-----\n");
98 Serial.flush(); // Ensure all serial data is printed before sleeping
99
100 // Start Deep Sleep
101 esp_deep_sleep_start();
102 }

```

2.4 Loop

The loop remains empty because the device resets automatically after waking from deep sleep.

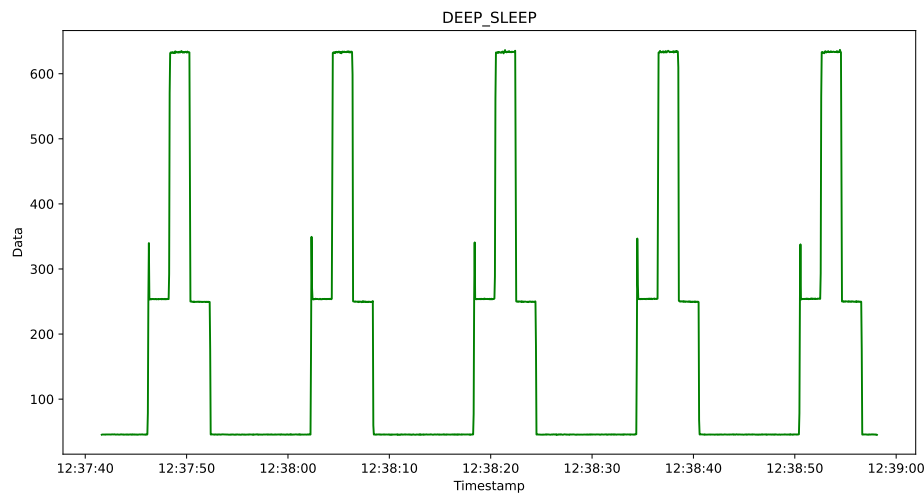
```

1 void loop() {
2 }

```

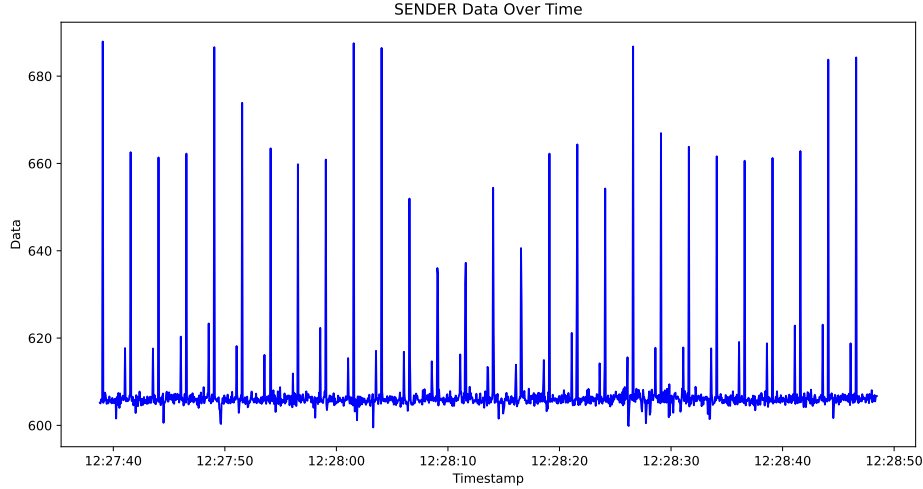
3 Power estimation

To figure out how much power our ESP32 actually uses, we started by analyzing the three CSV datasets provided for the project, which covered the deep sleep phase, booting, Wi-Fi on and Wi-Fi off phases, message transmission, and the sensing phase. We broke the system's operation down into specific tasks like booting up, reading the sensors, staying connected to Wi-Fi, and sending data, and then calculated the average power for each one by taking the mean of the data points showed in the plotted graphs. We started with the Deep Sleep graph:

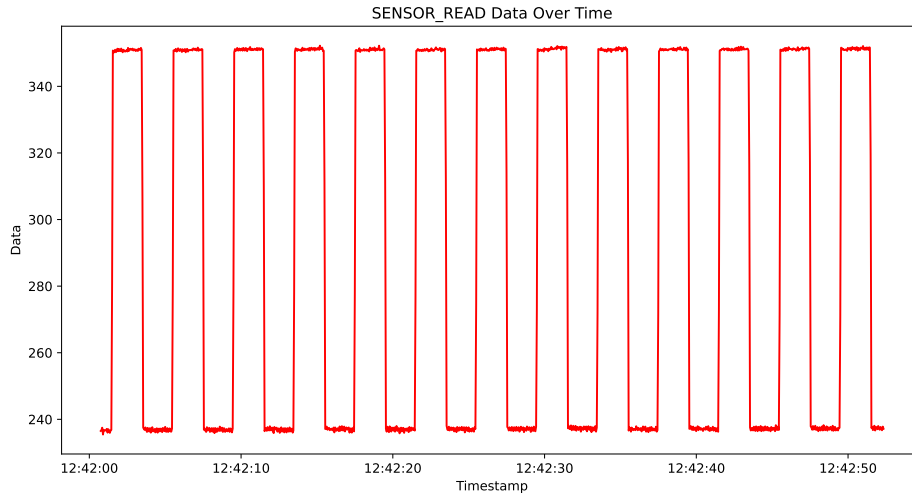


In our analysis of the power plot, we can clearly distinguish the various operational phases, including booting, a first part of the idle state, a Wi-Fi activation, a second part of the idle state, and a deep sleep. To measure the power consumption of each phase, we applied a threshold-based filtering method where specific "lines" served as boundaries to categorize the system's state. For instance, we isolated the deep sleep state by selecting all data points

with a power consumption below 100 mW and then calculated the arithmetic mean of those points to establish a representative value of 45.78 mW. We applied this same logic across the entire dataset: the Wi-Fi on phase was captured by filtering for points above 500 mW (averaging 632.04 mW), while the boot sequence and the idle phases were accurately separated into the 300–400 mW and 200–300 mW ranges, respectively. By using these distinct power brackets to segment the raw data, we were able to calculate precise mean values for every state. The same reasoning was applied to the other two files, which represented the power consumed during sensing and during transmission. Firstly we analyze the power consumption during the transmission phase:



In the sender plot, the distinct spikes represent active data transmissions at different signal strengths. To distinguish between them, we categorized all data points between 610 and 630 mW as representative of a low-power 2 dBm transmission, while points exceeding 640 mW were used to characterize the standard 19.5 dBm setting. This filtering process revealed that transmitting at 19.5 dBm draws a mean power of 667.41 mW, whereas reducing the signal to 2 dBm drops the average consumption to 617.88 mW. Finally the last graph remained to be analyzed was the power consumption of the sensor during the sensing phase:

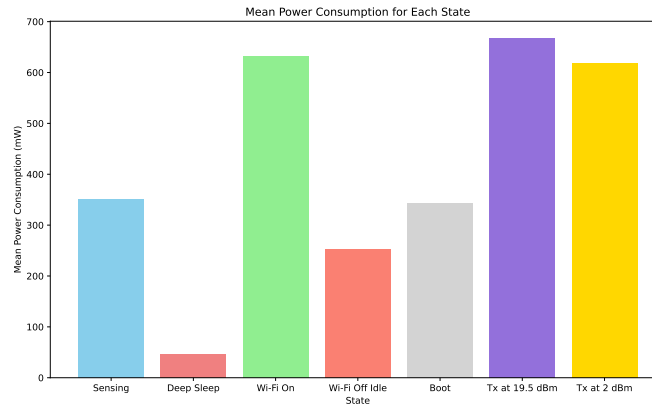


To determine the power consumption during the sensing phase, we filtered the dataset to isolate all data points with a power consumption exceeding 300 mW. The results highlighted a mean power consumption during Sensing phase of 350.16 mW. Below we summarize the power consumption of all the states:

Phase	Power (mW)
Boot	342.98
Sensing	350.16
WiFi ON	632.04
WiFi OFF	252.22
Transmission at 19.5 dBm	667.41
Transmission at 2 dBm	617.88
Deep sleep	45.78

Table 1: Analysis of power consumption in milliWatt (mW)

This analysis allowed us to build a complete power profile for the hardware, effectively creating a baseline for its electrical demands. By mapping out these states, we could see exactly what are, in general, the states requiring the highest power. The most demanding states, from the power consumption point of view, can be clearly seen by the following graph:



4 Energy consumption

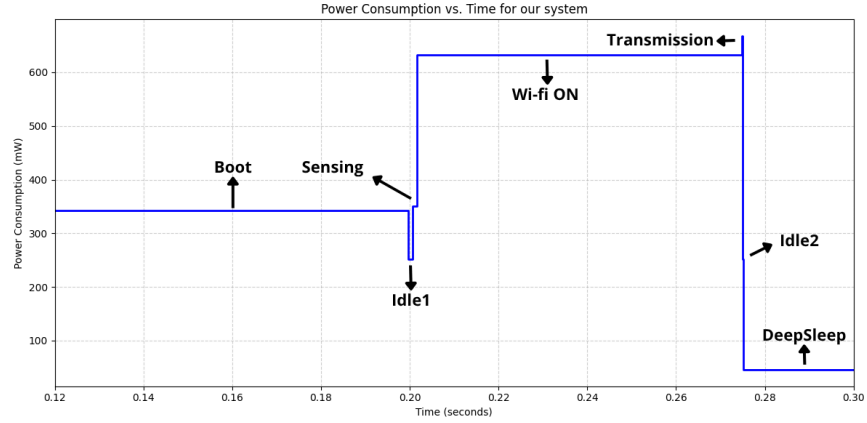
To account for the energy consumption of our system we simulated the system's functioning over 28 complete cycles to determine the precise duration of each operational state. Once the simulation finished, we computed the mean of the measurements take for each cycle, which resulted in these outputs:

Phase	Duration (μ s)
Boot	199,613.00
Sensing	1,029.43
WiFi on	73,206.75
Transmission	73.57
Total Idle	1,185.32
<i>Idle (Part 1)</i>	<i>1,038.86</i>
<i>Idle (Part 2)</i>	<i>146.46</i>
Deep sleep	1,400,000.00

Table 2: Details of the system's time durations

We obviously had a problem measuring the duration of the boot state, so we simply took the measurement of the boot phase from the DEEP_SLEEP csv file given to us for the project, which resulted in almost 200000 μ s of duration. Another minor issue came up with the measurement of the deep sleep phase which isn't possible over the Wokwi simulator we used, so the spec of the project gave us a fixed value calculated using the person code of the team leader. The timing data, extracted from our simulation, provided the exact duration in microseconds (μ s) for the dynamic phases of the cycle, which sequentially included: booting, a first idle phase, sensing, Wi-Fi on,

transmission, a second idle phase and finally a deep sleep phase. Once we've done that we plotted the power versus time graph, in which we can clearly see the power consumption of each state:

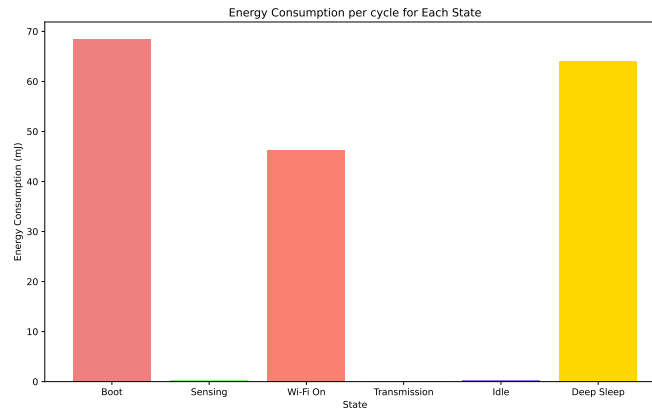


This power versus time graph illustrates the distinct operational phases of the system. The cycle initiates with the boot state, transitions into a primary idle phase, and subsequently enters the sensing phase. Following this, a significant spike in power consumption occurs, corresponding to the activation of the Wi-Fi. With the network module engaged, the system executes its data transmission. This is immediately followed by a brief, secondary idle period before the system finally enters a deep sleep phase to conserve energy. The graph's axes are deliberately zoomed in on this active window to clearly resolve these rapid state transitions. Then to get the energy consumption of each state we simply multiply the power consumption with the respective state duration. These are the results we obtained:

Phase	Energy (mJ)
Boot	68.46
Sensing	0.36
WiFi on	46.27
Transmission	0.05
Idle	0.30
Deep sleep	64.09

Table 3: Analysis of energy consumption in millijoules (mJ)

The above results are presented in the following histogram, to visualize them better:



The graph makes it clear for us that the main energy drain states are the booting, the wi-fi on and the deep sleep state.

4.1 Operating lifetime

To calculate the operating lifetime we need the battery energy and the total energy consumed per cycle. The first one is calculated using this formula $\text{Battery_energy} = 3859\%5000 + 15000$ that uses the last 4 digits of the person code of the team leader and the second one is calculated summing the values obtained in the previous paragraph which sums up to 0.18 J per cycle. Then we calculate the duty cycles in the following way:

$$\text{Duty Cycles} = \frac{E_{\text{battery}}}{E_{\text{tot}}} = 105,047 \text{ duty cycles} \quad (1)$$

The lifetime duration is calculated with:

$$T_{\text{tot}} = \frac{T_{\text{boot}} + T_{\text{sensing}} + T_{\text{wifi_on}} + T_{\text{tx}} + T_{\text{idle}} + T_{\text{deep_sleep}}}{10^6} \quad (2)$$

$$\text{Lifetime Duration} = \frac{\text{Duty Cycles} \cdot T_{\text{tot}}}{3600} = 48.879 \text{ hours} \quad (3)$$

This is a lifetime duration that is not feasible with respect to the field in which our system is going to be applied. Above we listed some improvements that must be made in order to increase the lifetime duration of the system.

5 Improvements

An analysis of the energy consumption histogram reveals that the Wi-Fi on phase, boot sequence, and deep sleep phase dominate the system's overall energy budget. A critical evaluation of these states highlights practical ideas for optimization.

5.1 Increasing deep sleep duration

Because the power requirements and duration of the boot sequence are intrinsic hardware constraints, optimizing this specific phase is unfeasible. Conversely, while the deep sleep state accounts for a significant portion of the total energy consumed, this is entirely due to its long duration rather than a high power draw. By extending the deep sleep period to 3 seconds, we decrease the frequency of active duty cycles. Lengthening this period of inactivity effectively reduces how often the system must execute the highly energy-intensive "boot" and transmission phases, leading to a lower average power consumption and extending the system's projected lifetime from 48.879 to 67.875 hours. This modification is highly feasible for our specific application, as it yields a sensing interval of approximately 3.5 to 4 seconds. For our system we assumed it was used inside a bathroom and given the typical nature of bathroom occupancy, this sampling resolution is more than adequate to ensure timely detection while providing a significant boost to the device's operational longevity.

```
deep_sleep_duration_2 = 3000000
Energy_deep_sleep_2 = (deep_sleep_duration_2 * deepsleep)/1000000
Tot_energy_2 = (Energy_boot + Energy_sensing + Energy_wifi_on + Energy_transmission + Energy_idle + Energy_deep_sleep_2)/1000
Duty_cycles_2 = Battery_energy / Tot_energy_2
Lifetime_duration_2 = Duty_cycles_2 * Tot_time
print(f"It will last for {Duty_cycles_2:.0f} duty cycles")
Tot_time_2 = (boot_duration + sensing_duration + wifi_on_duration + transmission_duration + idle_duration + deep_sleep_duration_2)/1000000
Lifetime_duration_2 = Duty_cycles_2 * Tot_time_2 / 3600
print(f"It will last for {Lifetime_duration_2:.3f} h")

It will last for 74609 duty cycles
It will last for 67.875 h
```

5.2 Lowering the power of the transmission

The second optimization we identified, has anything to do with the histogram derived by the energy consumption calculations, but we noticed that by reducing the transmission power to the minimum viable level of 2 dBm, from the 19.5 dBm initially utilized, the power consumption would decrease. As we previously mentioned we assumed the usage of our system is performed in a domestic bathroom, where the high-gain signals used are often unnecessary, as the physical distance between a sensor and its sink is relatively short. This is particularly true if the system leverages a hop-by-hop architecture, such as using smart bulbs as repeaters, which ensures that the transmission distance remains minimal. By applying this small change, we successfully reduced the power consumption in the

transmission-state from 667.41 mW to 617.88 mW. Although this represents a significant reduction in the power consumption specifically dedicated to the transmission state, the overall impact on the system's battery life remains modest. This is primarily because the duration of the transmission phase is extremely short compared to the rest of the duty cycle. The following calculations show the small improvement obtained over the lifetime duration of the system which passed from 48.879 hours to 48.880 hours.

```
Energy_transmission_2dBm = (transmission_duration * send2)/1000000
Tot_energy_2dBm = (Energy_boot + Energy_sensing + Energy_wifi_on + Energy_transmission_2dBm + Energy_idle + Energy_deep_sleep)/1000
Duty_cycles_2dBm = Battery_energy / Tot_energy_2dBm
print(f"It will last for {Duty_cycles_2dBm:.0f} duty cycles")
Tot_time = (boot_duration + sensing_duration + wifi_on_duration + transmission_duration + idle_duration + deep_sleep_duration)/1000000
Lifetime_duration_2dBm = Duty_cycles_2dBm*Tot_time/3600
print(f"It will last for {Lifetime_duration_2dBm:.3f} h")
```

It will last for 105049 duty cycles
It will last for 48.880 h

5.3 Shifting from periodic reporting to event-driven activation

The most significant enhancement to the system's power profile is to transmit the message only when a motion is detected and low luminosity is present. Basically we shift from traditional periodic reporting to event-driven activation. To quantify the impact of this logic, we modeled a realistic scenario where the sensor is placed in a bathroom that has a window which is well exposed to the sun. Under these conditions, the luminosity threshold (less than 100 lux) is typically only met during nighttime or early morning hours. Assuming a conservative estimate of 10 human interactions per day during these specific periods, the Wi-Fi module is only activated 10 times per day. Given that each duty cycle having the Wi-Fi on lasts approximately 73206.750 microseconds, the total daily active time for the Wi-Fi is calculated as:

$$\frac{73206.750 \times 10}{1,000,000} = 0.732 \text{ seconds}$$

Based on these initial conditions, where we assume a transmission at 19.5 dBm and a deep sleep interval of 1.4 seconds, the Wi-Fi module is operational for only 0.000847% of the system's total lifetime. Given that before the Wi-Fi was on at each cycle, it was enabled for 7690.149 seconds, meaning it was active for 4.37% of the time. In terms of energy saved, given that the Wi-Fi on phase consumes 632.04 mW, the energy consumption for the Wi-Fi on phase goes from 4860.5 J to 0.942 J for the system's lifetime. By restricting high-energy states to these specific, necessary moments, we have successfully decoupled the device's longevity from the clock and tied it directly to actual usage. The battery is no longer "powering a radio" but instead, it is powering a sensor that only calls upon the radio when relevant data must be sent, maximizing efficiency without compromising functionality. This logic is implemented in the improved code by adding this if guard:

```
1 if(motionDetected == HIGH && luminosity < 100) {
2   // part in which we turn on the Wi-Fi and send the message
3 }
```

After adding this we made sure using the simulator that the Wi-Fi on time and the transmission duration when no motion was detected and the luminosity was sufficient were 0, while became normal and a message was sent while motion and low luminosity were simulated. The simulation was successful as shown in the following output:

```

rst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc

--- TIMING BREAKDOWN (in microseconds) ---
Sensing duration: 1198
Wi-Fi On duration: 0
Transmission duration: 0
Idle duration (Wi-Fi OFF - Part 1): 1103
Idle duration (Wi-Fi OFF - Part 2): 17
Total Idle duration (Wi-Fi OFF): 1120
-----

ets Jul 29 2019 12:21:46

rst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
ets Jul 29 2019 12:21:46

rst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Message sent: MOTION_DETECTED-LUMINOSITY:4.16
Send Status: Ok

--- TIMING BREAKDOWN (in microseconds) ---
Sensing duration: 1004
Wi-Fi On duration: 224666
Transmission duration: 96
Idle duration (Wi-Fi OFF - Part 1): 1073
Idle duration (Wi-Fi OFF - Part 2): 18
Total Idle duration (Wi-Fi OFF): 1091
-----

```

Additionally, to mitigate the risk of a "silent failure", where the system stops to function without notifying the user of a need for intervention, we propose the implementation of a heartbeat mechanism. In our model, a prolonged absence of data is ambiguous, as it could signify either a lack of detected motion or a terminal hardware failure, such as battery exhaustion. To resolve this uncertainty, the system should be designed to transmit a signal every 100 boot cycles, regardless of sensor input. Unfortunately the Wokwi simulator does not currently support the persistence of variables across deep sleep states. In a physical deployment we would utilize the microcontroller's RTC (Real-Time Clock) memory. This specialized memory remains energized during sleep, allowing a boot counter to survive the power-down cycle. Although maintaining this persistent variable introduces a marginal increase in the deep sleep power consumption, this overhead is a necessary trade-off to provide the diagnostic visibility required to distinguish between a functional but idle system and a total malfunction.